

Python Notes

Vishwas K Singh



OPEN SOURCE
ACADEMIC NOTES



Contents

LICENSE	5
Summary of Rights	5
Under the Following Terms:	5
Full Legal Notice	5
Fundamentals of Python	7
Basics of programming	7
Types of programming languages	7
Python Programming Language	7
Executing python programs	8
Python Features	9
Understanding the execution of Python	9
How Python Manages memory	10
Data Types & Operators in Python	14
Variables	14
Input and Output in Python	16
Operators	24
Lists in Python	29
Dictionaries in Python	31
Tuples	32
Sets in Python	33
Control Flow Statements in Python	36
1. if Statement	36
2. for Loop	36
3. while Loop	37
4. break and continue	38
Functions in Python	39
Defining a Function	39
Calling a Function	39
Function Parameters and Return Values	39

Default Parameters	39
Keyword Arguments	40
Variable Length Arguments	40
Combining Different Types of Arguments	40
Structure of a Python Script	41
Scope of Variables	42
Command Line Arguments	44
Useful Built-in Functions	44
Modules & Packages in Python	47
Modules	47
Packages	47
Aliasing Modules & Imports	48
Exploring Modules with <code>help()</code> and <code>dir()</code>	48
Exploring Built-in Modules	49
The <code>string</code> Built-in Module	49
Exploring & Using the <code>math</code> Module	50
Error Handling & Debugging in Python	51
Error Handling	51
Debugging	52
File I/O in Python	56
Opening a File	56
Reading from a File	56
Writing to a File	56
Closing a File	57
Example: Copying a File	57
Summary	57
Object Oriented Programming (OOP) Concepts in Python	58
Key Concepts	58
Summary	60
Advanced OOPs Concepts	61
Multiple Inheritance	61

Polymorphism with Abstract Base Classes (ABCs)	61
Dunder Methods	63

Advanced Python Concepts	64
---------------------------------	-----------

Generators	64
Iterators	64
Decorators	65
Dataclasses	72
List Comprehension	73
Dictionary Comprehension	73
__slots__	73
Context Managers	74

LICENSE

Creative Commons Attribution 4.0 International License

Copyright (c) 2009-2026, Vishwas K Singh.

Summary of Rights

This is a human-readable summary of (and not a substitute for) the full legal text. You are free to:

Share — copy and redistribute the material in any medium or format. **Adapt** — remix, transform, and build upon the material for any purpose, even commercially.

Under the Following Terms:

Attribution — You must give appropriate credit to Vishwas K Singh, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No Additional Restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Full Legal Notice

Disclaimer of Warranties and Limitation of Liability:

The Licensor offers the Licensed Material as-is and as-available, and makes no representations or warranties of any kind concerning the Licensed Material, whether express, implied, statutory, or other. This includes, without limitation, warranties of title, merchantability, fitness for a particular purpose, non-infringement, absence of latent or other defects, accuracy, or the presence or absence of errors, whether or not known or discoverable.

In no event will the Licensor be liable to you on any legal theory (including, without limitation, negligence) or otherwise for any direct, special, indirect, incidental,

consequential, punitive, exemplary, or other losses, costs, expenses, or damages arising out of this Licensed Material or use of the Licensed Material.

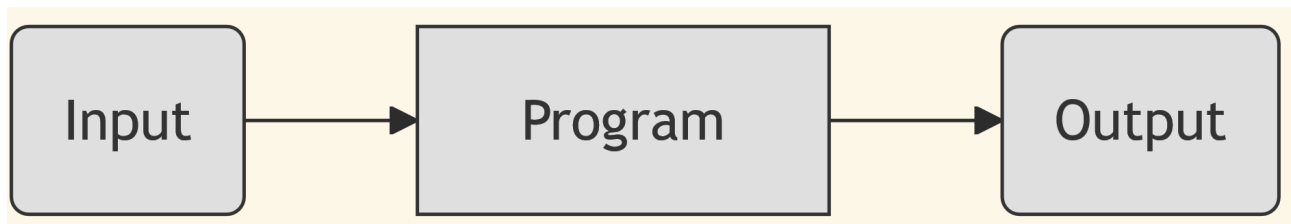
For the full legal code of this license, please visit: <https://creativecommons.org/licenses/by/4.0/legalcode>

Fundamentals of Python

Basics of programming

How do you approach a programming problem?

Think of a program as a black box which transforms an input to a specified output.



Researchers have found that more than 40% of the time is invested in coding in a overall project schedule. Finding logical solutions to problems in a language a computer understands means programing.

Types of programming languages

1. Compiled Languages: C, C++ etc..
Source code --> compile --> Binary Code --> execute
2. Interpreted languages: Python, Javascript, Java etc..
Source Code --> interpret(execute) Source Code --> Byte Code --> interpret

Python Programming Language

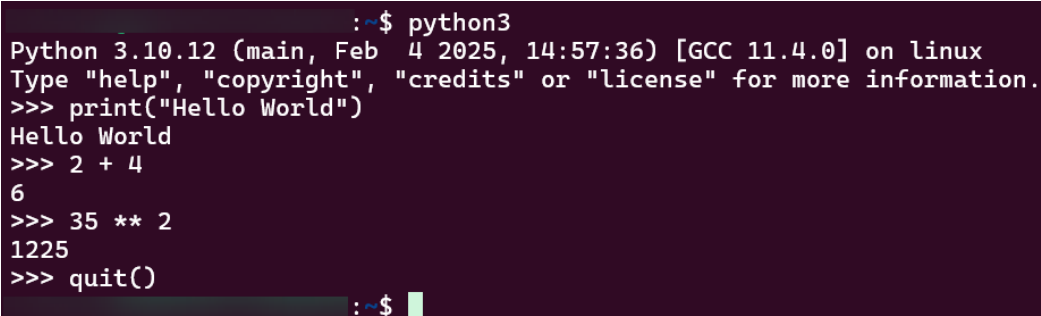
- Python is an **interpreted** language like Java
- It needs an interpreter to run
- It was invented by **Guido Van Rossum** in **1991**
- Current Version of python at the time of writing is **3.13.3**
- You need to install python Interpreter before you can execute python programs
- Interpretation means executing commands line by line
- It is a **platform-independent** language
- It is a **WORA - Write Once Run Anywhere** kind of language

- The python interpreter is tied to the OS architecture
- There are different kinds of python interpreters - **Jython, Cython, Pypi**

Executing python programs

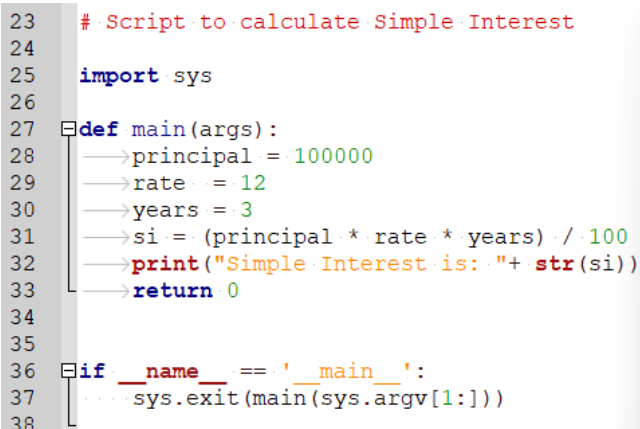
To execute python programs, we need python interpreter, it can be installed from python.org.

- Interactive mode of execution

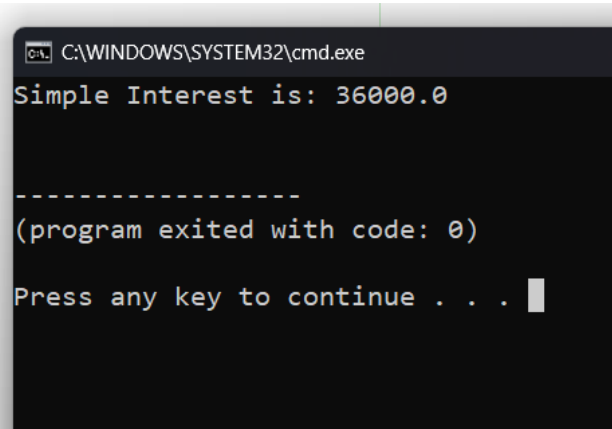


```
~$ python3
Python 3.10.12 (main, Feb 4 2025, 14:57:36) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>> 2 + 4
6
>>> 35 ** 2
1225
>>> quit()
~$
```

- Script Mode of execution



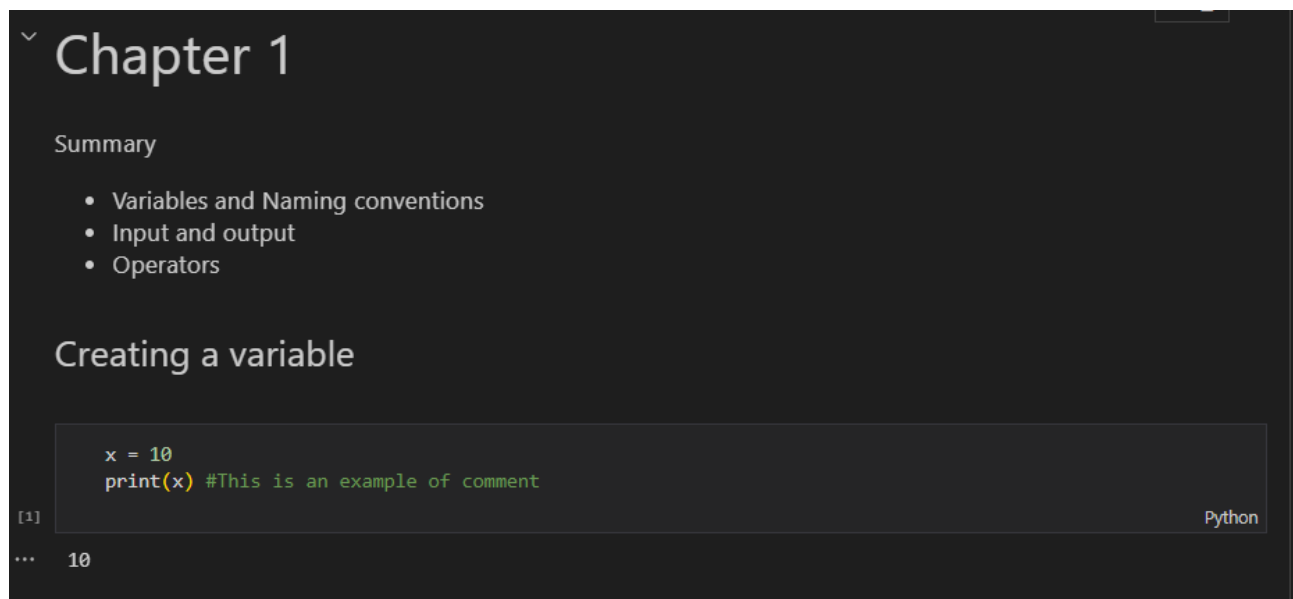
```
23 # Script to calculate Simple Interest
24
25 import sys
26
27 def main(args):
28     principal = 100000
29     rate = 12
30     years = 3
31     si = (principal * rate * years) / 100
32     print("Simple Interest is: " + str(si))
33     return 0
34
35
36 if __name__ == '__main__':
37     sys.exit(main(sys.argv[1:]))
38
```



```
C:\WINDOWS\SYSTEM32\cmd.exe
Simple Interest is: 36000.0

-----
(program exited with code: 0)
Press any key to continue . . .
```

- Notebook mode of execution



The screenshot shows a Jupyter Notebook interface. At the top, there's a section titled 'Chapter 1' with a dropdown arrow. Below it is a 'Summary' section containing a bulleted list: 'Variables and Naming conventions', 'Input and output', and 'Operators'. Further down is a section titled 'Creating a variable'. Below this is a code cell with the following Python code: `x = 10` and `print(x) #This is an example of comment`. The code cell is labeled '[1]' on the left and 'Python' on the right. Below the code cell, the output is shown as '10'.

Python Features

1. Portability
2. Object Oriented & Functional
3. Scalable
4. Interfacing with other languages & tools
5. Built on C API
6. Dynamic Programming
7. Ease of use & debugging

Understanding the execution of Python

- The execution of python programs involves 2 steps
 - Compilation
 - Interpreter
- Internally the program is converted to Byte Code & then the byte code is interpreted.
- we can explicitly compile to byte code

```
python -m py_compile <filename>.py
```

```
vishwas-:test_app$vim first.py
vishwas-:test_app$python -m py_compile first.py
vishwas-:test_app$ls
first.py  first.pyc
vishwas-:test_app$
```

- This file can be viewed using disassembly

```
python -m dis first.py
```

```
vishwas-:test_app$python -m dis first.py
1          0 LOAD_CONST          0 (10)
          3 STORE_NAME           0 (a)

2          6 LOAD_CONST          0 (10)
          9 STORE_NAME           1 (b)

3         12 LOAD_CONST          1 ('Sum ')
         15 LOAD_NAME             0 (a)
         18 LOAD_NAME             1 (b)
         21 BINARY_ADD
         22 BUILD_TUPLE           2
         25 PRINT_ITEM
         26 PRINT_NEWLINE
         27 LOAD_CONST          2 (None)
         30 RETURN_VALUE
```

- Next we can execute the byte code using the below command

```
python first.pyc
```

- There is a PVM (Python Virtual Machine) which executes the code line by line
- This is a time consuming process, hence there is a Just In Time (JIT) compiler which executes the program in a faster way(introduced recently).

How Python Manages memory

- Memory management in Python involves a private heap containing all Python objects and data structures.

- Management of this private heap is ensured internally by the Python memory manager.
- A raw memory allocator ensures that there is enough room in the private heap for storing all Python-related data by interacting with the memory manager of the operating system.
- The allocation of heap space for Python objects and other internal buffers is performed on demand by the Python memory manager through the Python/C API functions
- Python handles memory management automatically using mechanisms like reference counting and garbage collection

Reference Counting

It is one of the primary memory management techniques used in Python, where:

- Every object keeps a reference counter, which tells how many variables (or references) are currently pointing to that object.
- When a new reference to the object is created, counter increases.
- When a reference is deleted or goes out of scope, counter decreases.
- If the counter reaches zero, it means no variable is using the object anymore, so Python automatically deallocates (frees) that memory.

Garbage Collection

It is a process in which Python automatically frees memory occupied by objects that are no longer in use.

- If an object has no references pointing to it (i.e., nothing is using it), garbage collector removes it from memory.
- This ensures that unused memory can be reused for new objects.
- Python provides the gc module to interact with and fine-tune the collector's behavior for specific performance needs or debugging.

How Generational GC Works

The generational garbage collector organizes objects into three "generations" based on their age (how long they have survived previous collections):

- **Generation 0:** Contains newly created objects. It is collected most frequently because most objects are short-lived ("die young").
- **Generation 1:** Contains objects that survived a Generation 0 collection. It is collected less frequently than Generation 0.
- **Generation 2:** Contains long-lived objects that survived Generation 1 collections. It is collected the least frequently.

The garbage collector runs automatically when the number of object allocations minus deallocations exceeds a predefined threshold for each generation.

Python Arenas

- **Arenas** are the foundational, largest, contiguous chunks of memory that Python allocates from the operating system to manage the Python private heap.
- They are primarily used by the Object and Mem allocation domains to store Python objects (like integers, strings, lists, etc.) and small buffers.
- The main purpose of using arenas is to minimize the number of calls to the system's general-purpose memory allocator (malloc), which is a relatively expensive operation. By requesting a large chunk (an arena) all at once, Python can handle object allocation and deallocation internally and much more efficiently.
- The arena system organizes the Python private heap in a three-tiered structure
- Arenas:
 - An arena is a large block of memory, often around 256KB (the size may vary depending on the Python version and platform).
 - Python allocates an arena from the OS when it needs more memory for objects and buffers.
 - Once an arena is allocated, Python manages the memory within it internally.
- Pools (Pages)

- Each arena is divided into multiple fixed-size sections called pools (or pages).
- A pool is typically 4KB in size.
- Pools are categorized based on the size of the objects they will hold (e.g., a pool might be designated to hold only 8-byte objects).
- Blocks (The Object Holder)
- Each pool is further subdivided into small, uniform-sized blocks.
- A block is the smallest unit of memory that Python uses to hold a single Python object (e.g., an integer object, a list header, a small string).
- All blocks within a given pool are the exact same size. This fixed-size block allocation makes it incredibly fast to find a free spot for a new object and to reuse the space once an object is deallocated.

Data Types & Operators in Python

Python has 5 basic data types:

Data Type	Description	Example
number	Any integer or float	12, -123, -56.89
string	anything enclosed in single or double quotes	'abcd', "HP"
list	dynamic array	[1, 2, "abd"]
Tuple	static array	(1, 2, 3)
Dictionary	Key value pairs	{'eno': 101}

Variables

- The smallest element that can hold data.
- Depending on the value stored, the type of variable changes accordingly.
- It represents a memory location.
- It can be any word other than the keywords.
- = operator is used to assign value to a variable. To create a variable
- **Note:**
 1. Python internally caches small integers (from -5 to 256). So small integers will point to same object

```
>>> x = 100
>>> y = 100
>>> x is y
True
>>> id(x)
140057005315536
>>> id(y)
140057005315536
>>> z = 296
>>> m = 296
>>> id(z)
140057004112848
>>> id(m)
140057004112720
>>> █
```

Syntax:

```
var_name = value
```

Rules for creating variables:

1. no declaration (only assignment)
2. case sensitive
3. Variables created in a script are global
4. A variable is represented as id in the memory, to know we can use `id()` method

```
>>> ename = "John Doe"
>>> type(ename)
<class 'str'>
>>> ename
'John Doe'
>>> id(ename)
139704689004080
>>> emp_name = ename
>>> emp_name
'John Doe'
>>> id(emp_name)
139704689004080
>>> del(ename)
>>> emp_name
'John Doe'
>>> id(emp_name)
139704689004080
>>> █
```

Input and Output in Python

Python provides simple functions for input and output (I/O) operations.

Output with `print()`

The `print()` function displays data to the standard output (usually the screen). You can print multiple values, use separators, and change the end character.

Python offers multiple ways to format strings when printing variables and values.

1. Using Format Specifiers with `format()`

The `str.format()` method allows you to insert values into a string using curly braces `{}` as placeholders.

2. Using f-Strings (Formatted String Literals)

Introduced in Python 3.6, f-strings provide a concise way to embed expressions inside string literals, using `{}`.

```
1#!/usr/bin/env python3
2# Author: Vishwas K Singh
3# Email: vishwask.reach@gmail.com
4# Script to demonstrate various printing options in python
5
6# Printing the variables
7print("Hello, World!")
8name = "Alice"
9age = 25
10pi = 3.14159
11print("Hello,", name, "age:", age)
12
13print("A", "B", "C", sep="-")
14print("Done", end="!")
15print() # To make the following in new line
16
17# C-like printing
18print("Name: %s, Age: %i"%(name, age))
19print("Name: {}, Age: {}".format(name, age))
20
21print("Name: {0}, Age: {1}".format(name, age))
22print("Name: {n}, Age: {a}".format(n=name, a=age))
23print("Value of pi: {:.2f}".format(pi))
24
25# f-strings
26print(f"Value of pi: {pi:.2f}")
27print(f"Next year, {name} will be {age + 1} years old.")
28
```

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.194:
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: =====
Hello, World!
Hello, Alice age: 25
A-B-C
Done!
Name: Alice, Age: 25
Name: Alice, Age: 25
Name: Alice, Age: 25
Name: Alice, Age: 25
Value of pi: 3.14
Value of pi: 3.14
Next year, Alice will be 26 years old.

>>>
```

Input in Python

1. Input with `input()` The `input()` function reads a line of text from the user as a string.

Example:

```
name = input("Enter your name: ")
print("Hello,", name)
```

To read numbers, convert the input string:

```
age = int(input("Enter your age: "))
price = float(input("Enter price: "))
```

Input Using `sys.stdin`

Besides the built-in `input()` function, Python allows reading input using the `sys` module, which provides access to standard input via `sys.stdin`. This is useful for reading multiple lines or handling input more efficiently, especially in competitive programming or when processing large amounts of data.

Example:

```
import sys

# Read a single line
line = sys.stdin.readline()
print("You entered:", line.strip())

# Read all lines as a list
lines = sys.stdin.readlines()
for l in lines:
    print(l.strip())
```

- `sys.stdin.readline()` reads one line at a time (including the newline character).
- `sys.stdin.readlines()` reads all lines until EOF and returns a list.

Note: When using `sys.stdin`, input is typically provided via files or redirected input, not interactively.

Working with numbers:

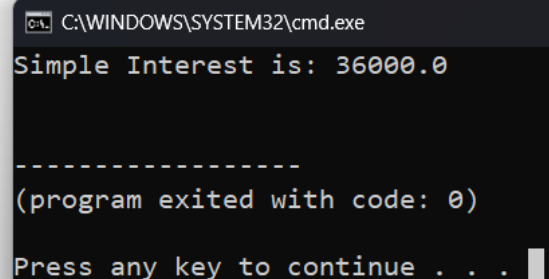
```
>>> x = 123
>>> type(x)
<class 'int'>
>>> id(x)
140293271359536
>>> z = float(x)
>>> x
123
>>> z
123.0
>>> y = 5
>>> x + y
128
>>> x - y
118
>>> x * y
615
>>> x / y
24.6
>>> x // y Integer Division
24
>>> x % y
3
>>> x ** y x to the power y
28153056843
>>>
```

Example Programs

```
principal = 100000
rate = 12
years = 3
si = (principal * rate * years) / 100
print("Simple Interest is: " + str(si))
```

To Calculate the Simple Interest Output

```
23 # Script to calculate Simple Interest
24
25 import sys
26
27 def main(args):
28     principal = 100000
29     rate = 12
30     years = 3
31     si = (principal * rate * years) / 100
32     print("Simple Interest is: " + str(si))
33     return 0
34
35
36 if __name__ == '__main__':
37     sys.exit(main(sys.argv[1:]))
38
```



C:\WINDOWS\SYSTEM32\cmd.exe

Simple Interest is: 36000.0

(program exited with code: 0)

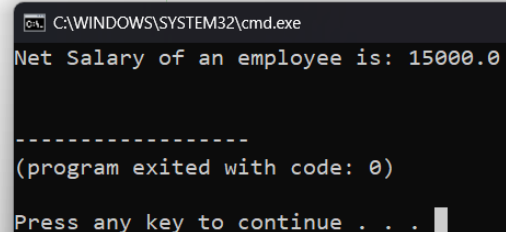
Press any key to continue . . .

```
basic = 10000
hra = .4 * basic
da = .2 * basic
gross = (basic + hra + da)
tax = .1 * basic
net = gross - tax

print("Net Salary of an employee is: " + str(net))
```

Computing net salary of an employee Output

```
22 # Script to calculate net salary of an employee
23
24
25 import sys
26
27
28 def main(args):
29     basic = 10000
30     hra = .4 * basic
31     da = .2 * basic
32     gross = (basic + hra + da)
33     tax = .1 * basic
34     net = gross - tax
35
36     print("Net Salary of an employee is: " + str(net))
37     return 0
38
```



C:\WINDOWS\SYSTEM32\cmd.exe

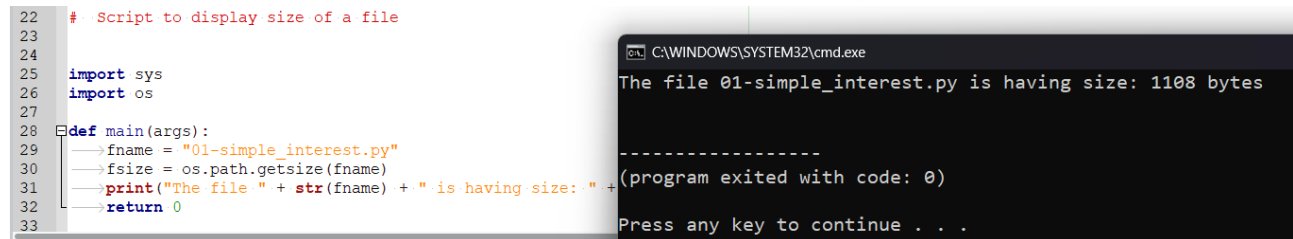
Net Salary of an employee is: 15000.0

(program exited with code: 0)

Press any key to continue . . .

```
fname = "01-simple_interest.py"
fsize = os.path.getsize(fname)
print("The file " + str(fname) + " is having size: " + str(fsize) +
      " bytes")
```

Displaying the size of a file Output



```
22 # Script to display size of a file
23
24
25 import sys
26 import os
27
28 def main(args):
29     fname = "01-simple_interest.py"
30     fsize = os.path.getsize(fname)
31     print("The file " + str(fname) + " is having size: " + str(fsize) +
32           " bytes")
33     return 0
```

C:\WINDOWS\SYSTEM32\cmd.exe
The file 01-simple_interest.py is having size: 1108 bytes

(program exited with code: 0)
Press any key to continue . . .

Working with strings

Anything which is enclosed in single or double quotes is a string. Multi-line strings are created using three single quotes or 3 double quotes at the beginning & at the end. A string is unlimited & immutable (read-only).

Example: 'abcd', "HP"

```
>>> x = 'computer'
>>> type(x)
<class 'str'>
>>> id(x)
139905856163824
>>> len(x)
8
>>> max(x
... )
'u'
>>> min(x)
'c'
>>> x[0]
'c'
>>> x[3]
'p'
>>> x[-1]
'r'
>>> x[-4]
'u'
>>> █
```

String Slicing String slicing in Python allows you to extract a portion (substring) of a string using the syntax `string[start:stop:step]`. The start index is inclusive, stop is exclusive, and step determines the stride. If omitted, defaults are `start=0`, `stop=len(string)`, and `step=1`.

Example:

```
text = "Hello, World!"
print(text[0:5])      # Output: Hello
print(text[7:])       # Output: World!
print(text[::-1])     # Output: !dlrow ,olleH
```

```
>>> x = 'computer'
>>> x[2:]
'mputer'
>>> x[:2]
'co'
>>> x[-3:]
'ter'
>>> x[:-3]
'compu'
>>> x[2:5]
'mpu'
>>> x[2:3]
'm'
>>> x[2:-3]
'mpu'
>>> x[:]
'computer'
>>> x[::2]
'cmue'
>>> x[::-1]
'retupmoc'
>>> x[::-2]
'rtpo'
>>> x[1::2]
'optr'
>>> █
```

Typecasting

Typecasting is the process of converting a value from one data type to another. This is often necessary when you want to perform operations between different types, or when a function expects a specific type as input.

There are two main types of typecasting:

Implicit Typecasting (Automatic Conversion):

The programming language automatically converts one data type to another. Example: In Python, adding an integer and a float will automatically convert the integer to a float.

Explicit Typecasting (Manual Conversion):

The programmer manually converts one data type to another using casting functions or syntax. Example: `float(5)` converts the integer 5 to a float 5.0.

Why is typecasting important?

Prevents type errors in your code. Ensures data is in the correct format for operations or function calls. Helps with interoperability between different parts of a program. Gotcha: Be careful when typecasting, as converting between incompatible types (e.g., a string that doesn't represent a number to an integer) can cause runtime errors. Always validate or handle exceptions when typecasting user input.

```
>>> a = 123
>>> type(a)
<class 'int'>
>>> b = '456'
>>> type(b)
<class 'str'>
>>> = a + b
      File "<stdin>", line 1
        = a + b
IndentationError: unexpected indent
>>> c = a + b
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> c = a + int(b)
>>> d = str(a) + b
>>> c
579
>>> d
'123456'
>>> █
```

Additional Concepts

```
>>> a = 1
>>> chr(a)
'\x01'
>>> b = '1'
>>> ord(b)
49
>>> ord('a')
97
>>> 'a' + 'b'
'ab'
>>> 'a' * 12
'aaaaaaaaaaaa'
>>> '-' * 12
'-----'
>>>
```

byte value of an integer

ASCII Value of a character

Adding 2 characters concatenates them

Multiply a str by integer repeats the character

```
>>> chr(65)
'A'
>>> chr(49)
'1'
>>>
```

Operators

Python Operators

Operators are special symbols or keywords in Python that are used to perform operations on variables and values. Python supports several types of operators:

Arithmetic Operators

Used for mathematical operations:

Operator	Description	Example	Result
+	Addition	2 + 3	5
-	Subtraction	5 - 2	3
*	Multiplication	3 * 4	12
/	Division	10 / 2	5.0
//	Floor Division	7 // 2	3
%	Modulus	7 % 2	1

Operator	Description	Example	Result
<code>**</code>	Exponentiation	<code>2 ** 3</code>	8

Comparison Operators

Used to compare values, returning True or False:

Operator	Description	Example	Result
<code>==</code>	Equal to	<code>3 == 3</code>	True
<code>!=</code>	Not equal to	<code>3 != 4</code>	True
<code>></code>	Greater than	<code>5 > 2</code>	True
<code><</code>	Less than	<code>2 < 5</code>	True
<code>>=</code>	Greater or equal	<code>5 >= 5</code>	True
<code><=</code>	Less or equal	<code>2 <= 3</code>	True

Assignment Operators

Used to assign values to variables:

Operator	Example	Equivalent to
<code>=</code>	<code>a = 5</code>	
<code>+=</code>	<code>a += 2</code>	<code>a = a + 2</code>
<code>-=</code>	<code>a -= 2</code>	<code>a = a - 2</code>
<code>*=</code>	<code>a *= 3</code>	<code>a = a * 3</code>
<code>/=</code>	<code>a /= 2</code>	<code>a = a / 2</code>
<code>//=</code>	<code>a //= 2</code>	<code>a = a // 2</code>
<code>%=</code>	<code>a %= 2</code>	<code>a = a % 2</code>
<code>**=</code>	<code>a **= 2</code>	<code>a = a ** 2</code>

Logical Operators

Used to combine conditional statements:

Operator	Description	Example	Result
and	Logical AND	True and False	False
or	Logical OR	True or False	True
not	Logical NOT	not True	False

Membership Operators

Test for membership in a sequence:

Operator	Description	Example	Result
in	Value present	'a' in 'cat'	True
not in	Value not present	'x' not in 'cat'	True

Identity Operators

Compare memory locations of two objects:

Operator	Description	Example	Result
is	Same object	a is b	True/False
is not	Not same object	a is not b	True/False

Bitwise Operators

Operate on binary representations:

Operator	Description	Example	Result
&	AND	5 & 3	1
	OR	5 3	7

Operator	Description	Example	Result
<code>^</code>	XOR	<code>5 ^ 3</code>	6
<code>~</code>	NOT	<code>~5</code>	-6
<code><<</code>	Left Shift	<code>5 << 1</code>	10
<code>>></code>	Right Shift	<code>5 >> 1</code>	2

Example:

```
a = 10
b = 3
print(a + b)      # 13
print(a > b)      # True
print(a & b)       # 2
print(a is b)     # False
```

```
>>> a = 10
>>> b = 3
>>> print( a + b)
13
>>> print(a > b)
True
>>> print(a & b)
2
>>> print( a is b)
False
>>> █
```

Operators are fundamental to writing expressions and controlling the flow of your Python programs.

Misc Concepts

```
>>> 1 and 0
0
>>> 0 and 2
0
>>> 2 and 6
6
>>> 5 and 3
3
>>> 1 or 0
1
>>> 0 or 2
2
>>> 5 or 3
5
>>> 9 or 8
9
>>> █
```

Short Circuit Evaluation

Short circuit evaluation refers to the way logical operators `and` and `or` work in Python. When evaluating expressions, Python stops as soon as the result is determined:

- For `and`, if the first operand is `False`, the whole expression is `False` and the second operand is not evaluated.
- For `or`, if the first operand is `True`, the whole expression is `True` and the second operand is not evaluated.

Example:

```
def check():
    print("Function called")
    return True

# 'check()' is not called because the first condition is False
if False and check():
    print("Won't print")
```

```
# 'check()' is called because the first condition is True
if True or check():
    print("Will print")
```

Guard Pattern The guard pattern uses short circuit evaluation to prevent errors by ensuring that certain conditions are met before executing code that could fail. It's commonly used to check for None or validate input before accessing attributes or methods.

Example:

```
user = None

# Guard pattern: Only access 'name' if 'user' is not None
if user is not None and user.name == "Alice":
    print("Hello, Alice!")
```

Here, `user.name` is only accessed if `user` is not `None`, preventing an `AttributeError`.

Short circuiting and guard patterns help write safer and more efficient code.

```
>>> x = 6
>>> y = 2
>>> x / y
3.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
3.0
>>> █
```

Lists in Python

A **list** is a built-in data type in Python used to store an ordered collection of items, which can be of different types (integers, strings, other lists, etc.). Lists are mutable,

meaning you can change their contents after creation.

Creating a List:

```
numbers = [1, 2, 3, 4, 5]
mixed = [1, "apple", 3.14, [2, 4]]
```

Accessing Elements:

- Use zero-based indexing: `numbers[0]` gives 1
- Negative indices count from the end: `numbers[-1]` gives 5

Modifying Lists:

```
numbers[2] = 10      # Change value at index 2
numbers.append(6)     # Add an item to the end
numbers.insert(1, 7)  # Insert 7 at index 1
del numbers[0]        # Remove item at index 0
```

Common List Methods:

Method	Description	Example
<code>append(x)</code>	Add item to end	<code>numbers.append(10)</code>
<code>insert(i, x)</code>	Insert item at position <code>i</code>	<code>numbers.insert(1, 20)</code>
<code>remove(x)</code>	Remove first occurrence of <code>x</code>	<code>numbers.remove(3)</code>
<code>pop([i])</code>	Remove and return item at index <code>i</code>	<code>numbers.pop()</code>
<code>sort()</code>	Sort the list in place	<code>numbers.sort()</code>
<code>reverse()</code>	Reverse the list in place	<code>numbers.reverse()</code>

Iterating Over a List:

```
for item in numbers:
    print(item)
```

Lists are one of the most versatile and commonly used data structures in Python.

List Slicing

List slicing in Python allows you to extract a portion (sublist) of a list using the syntax `list[start:stop:step]`. The start index is inclusive, stop is exclusive, and step determines the stride. If omitted, defaults are `start=0`, `stop=len(list)`, and `step=1`.

Example:

```
numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1:4])    # [20, 30, 40]
print(numbers[:3])     # [10, 20, 30]
print(numbers[::2])    # [10, 30, 50]
print(numbers[::-1])   # [60, 50, 40, 30, 20, 10]
```

Slicing is a powerful way to access and manipulate sublists without modifying the original list.

Dictionaries in Python

A **dictionary** is a built-in data type in Python that stores data as key-value pairs. Dictionaries are **unordered** (prior to Python 3.7), mutable, and indexed by keys, which must be immutable types (like strings, numbers, or tuples).

- The keys in a dictionary are Unordered, Immutable, Unique **Creating a Dictionary:**

```
student = {
    "name": "Alice",
    "age": 20,
    "marks": [85, 90, 92]
}
```

Accessing Values:

- Use the key inside square brackets: `student["name"]` gives "Alice"

- Use `.get()` to avoid errors if the key is missing: `student.get("grade")` returns `None`

Modifying Dictionaries:

```
student["age"] = 21           # Update value
student["grade"] = "A"       # Add new key-value pair
del student["marks"]         # Remove a key-value pair
```

Common Dictionary Methods:

Method	Description	Example
<code>keys()</code>	Returns all keys	<code>student.keys()</code>
<code>values()</code>	Returns all values	<code>student.values()</code>
<code>items()</code>	Returns all key-value pairs as tuples	<code>student.items()</code>
<code>get(key)</code>	Returns value for key or <code>None</code>	<code>student.get("name")</code>
<code>update(dict2)</code>	Updates dictionary with another dictionary	<code>student.update({"name": "Vishwas"})</code>
<code>pop(key)</code>	Removes key and returns its value	<code>student.pop("grade")</code>

Iterating Over a Dictionary:

```
for key, value in student.items():
    print(key, value)
```

Dictionaries are ideal for representing structured data and for fast lookups by key.

Tuples

A **tuple** is a built-in data type in python used to store a sequence of items. They are **immutable**. Once created they cannot be modified.

```
# Creating a tuple
t1 = (1,2,3,4)
print(f"The type of t1: {t1} is {type(t1)}")
# We can also use without parenthesis
# We are compressing multiple values into single variable
t2 = 1,2,3,4
print(f"The type of t2: {t2} is {type(t2)}")
# We can extract the same
a,b,c,d = t2 # The number of variables must be same as number of
             elements
print(f"The value of a: {a}")
print(f"The value of b: {b}")
print(f"The value of c: {c}")
print(f"The value of d: {d}")
# We can compare the tuples
print(f"Are the tuples equal: {t1 == t2}")
t3 = 1,2,3,5
print(f"Is t1 > t3 : {t1 > t2}")
print(f"Is t1 < t3 : {t1 < t2}")
```

- Until there are same / equal elements the comparison continues.
- Once they are different the comparison is evaluated & result is put.

Sets in Python

A **set** is a built-in data type in Python used to store an unordered collection of unique items. Sets are mutable, but their elements must be immutable types (like numbers, strings, or tuples).

Creating a Set:

```
fruits = {"apple", "banana", "cherry"}
empty_set = set() # Note: {} creates an empty dictionary, not a
                  set
```

Key Properties:

- No duplicate elements: Each item appears only once.

- Unordered: No indexing or slicing.
- Mutable: You can add or remove elements.

Common Set Methods:

Method	Description	Example
<code>add(x)</code>	Add element x to the set	<code>fruits.add("orange")</code>
<code>remove(x)</code>	Remove element x (error if absent)	<code>fruits.remove("banana")</code>
<code>discard(x)</code>	Remove element x (no error)	<code>fruits.discard("pear")</code>
<code>pop()</code>	Remove and return an arbitrary item	<code>fruits.pop()</code>
<code>clear()</code>	Remove all elements	<code>fruits.clear()</code>
<code>copy()</code>	Return a shallow copy	<code>fruits.copy()</code>

Set Operations:

Sets support mathematical operations like union, intersection, difference, and symmetric difference.

Operation	Syntax	Example Result
Union	<code>set1 set2</code>	Items in both sets
Intersection	<code>set1 & set2</code>	Items in set1 not in set2
Difference	<code>set1 - set2</code>	Items in either, not both
Symmetric Diff.	<code>set1 ^ set2</code>	

Example:

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)  # {1, 2, 3, 4, 5}
print(a & b)  # {3}
print(a - b)  # {1, 2}
print(a ^ b)  # {1, 2, 4, 5}
```

Sets are useful for membership testing, removing duplicates, and performing set-theoretic operations.

Control Flow Statements in Python

Control flow statements allow you to control the execution order of statements in your code. The main types are: `if`, `for`, and `while`.

1. `if` Statement

Used for conditional execution.

```
x = 10
if x > 5:
    print("x is greater than 5")
elif x == 5:
    print("x is 5")
else:
    print("x is less than 5")
```

2. `for` Loop

Used to iterate over a sequence.

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Using the `range()` Function

The `range()` function generates a sequence of numbers, commonly used with `for` loops.

Syntax:

`range(stop)`

`range(start, stop)`

`range(start, stop, step)`

- start: Starting value (default is 0)
- stop: End value (not included)
- step: Increment (default is 1)

Example:

```
for i in range(3):  
    print(i)  # Outputs 0, 1, 2  
  
for i in range(1, 6, 2):  
    print(i)  # Outputs 1, 3, 5
```

Decremental Loop Example

You can use a for loop with a negative step in range () to count down.

```
for i in range(5, 0, -1):  
    print(i)  # Outputs 5, 4, 3, 2, 1
```

3. while Loop

Repeats as long as a condition is true.

```
count = 0  
while count < 3:  
    print("Count:", count)  
    count += 1
```

Infinite Loops

An infinite loop is a loop that never ends because its condition always evaluates to True. Infinite loops are useful in situations where you want your program to keep running until it is explicitly stopped, such as in servers or waiting for user input.

Example:

```
while True:
    print("This will run forever unless stopped manually.")
```

Be careful with infinite loops, as they can cause your program to become unresponsive if not handled properly. Always ensure there is a way to break out of the loop when needed.

4. break and continue

- `break` exits the loop.
- `continue` skips to the next iteration.

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Note: Though `for` loop and `while` loop can be used to handle looping, the rule of thumb is **When you know the number of steps (iterations) to take, use `for` loop otherwise (When you don't know the number of steps or you know the exit condition) use `while` loop**

Functions in Python

Functions are reusable blocks of code that perform a specific task. They help organize code, avoid repetition, and improve readability.

Defining a Function

Use the `def` keyword to define a function:

```
def greet(name):  
    print(f"Hello, {name}!")
```

Calling a Function

Invoke a function by using its name and passing required arguments:

```
greet("Alice")
```

Function Parameters and Return Values

Functions can accept parameters and return values:

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)  # result is 8
```

Default Parameters

You can provide default values for parameters:

```
def greet(name="World"):
    print(f"Hello, {name}!")
```

Keyword Arguments

Arguments can be passed by name:

```
greet(name="Bob")
```

Variable Length Arguments

Functions can accept a variable number of arguments using `*args` for positional arguments and `**kwargs` for keyword arguments:

```
def print_numbers(*args):
    for number in args:
        print(number)

print_numbers(1, 2, 3)  # prints 1, 2, 3

def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=30)
```

Combining Different Types of Arguments

You can combine fixed (positional), default, variable-length positional (`*args`), and variable-length keyword (`**kwargs`) arguments in a single function. The order should be:

1. Positional arguments
2. Default arguments

3. *args
4. **kwargs

```
def example(a, b=2, *args, **kwargs):  
    print(f"a: {a}")  
    print(f"b: {b}")  
    print(f"args: {args}")  
    print(f"kwargs: {kwargs}")  
  
example(1, 3, 4, 5, x=10, y=20)
```

Output:

```
a: 1  
b: 3  
args: (4, 5)  
kwargs: {'x': 10, 'y': 20}
```

This allows your function to handle a flexible number and type of arguments.

Structure of a Python Script

A typical Python script consists of the following components:

1. **Shebang Line (optional):** Indicates the interpreter to use when running the script from the command line.

```
#!/usr/bin/env python3
```

2. **Module Imports:** Import necessary modules or packages at the top of the script.

```
import sys  
import math
```

3. **Global Variables and Constants:** Define any constants or global variables.

```
PI = 3.14159
```

4. **Function and Class Definitions:** Define reusable functions and classes.

```
def calculate_area(radius):  
    return PI * radius ** 2
```

5. **Main Execution Block:** Use the `if __name__ == "__main__":` construct to specify code that should run when the script is executed directly.

```
if __name__ == "__main__":  
    print(calculate_area(5))
```

This structure helps organize code, improves readability, and allows code reuse.

Scope of Variables

The scope of a variable determines where it can be accessed within your code.

Local Scope

Variables defined inside a function are local to that function and cannot be accessed outside:

```
def my_function():  
    x = 10 # local variable  
    print(x)  
  
my_function()  
# print(x) # This would cause an error: NameError: name 'x' is not  
# defined
```

Global Scope

Variables defined outside any function are global and can be accessed throughout the script:

```
y = 20 # global variable

def show_y():
    print(y)

show_y() # prints 20
```

The global Keyword

To modify a global variable inside a function, use the `global` keyword:

```
count = 0

def increment():
    global count
    count += 1

increment()
print(count) # prints 1
```

The nonlocal Keyword

The `nonlocal` keyword allows you to modify a variable in an enclosing (but non-global) scope, such as inside nested functions:

```
def outer():
    x = 5
    def inner():
        nonlocal x
        x = 10
    inner()
    print(x) # prints 10

outer()
```

Understanding variable scope helps prevent bugs and makes your code easier to maintain.

Command Line Arguments

Python scripts can accept input from the command line using the `sys.argv` list from the `sys` module. This allows you to pass arguments to your script when you run it.

- `sys.argv` is a list where:
 - The first element (`sys.argv[0]`) is the script name.
 - The following elements are the arguments passed to the script.

Example:

Suppose you have a script called `example.py`:

```
import sys

print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])
```

If you run the script from the command line:

```
python example.py hello world
```

Output:

```
Script name: example.py
Arguments: ['hello', 'world']
```

You can use these arguments to control your script's behavior, such as processing files or setting options. For more advanced argument parsing, consider using the `argparse` module.

Useful Built-in Functions

Python provides several built-in functions that are commonly used when working with sequences and iterables:

zip

The `zip` function combines two or more iterables (like lists or tuples) element-wise into tuples:

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

Output:

```
Alice: 85
Bob: 92
Charlie: 78
```

enumerate

The `enumerate` function adds a counter to an iterable, returning pairs of index and value:

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output:

```
0 apple
1 banana
2 cherry
```

sorted

The `sorted` function returns a new sorted list from the items in any iterable:

```
numbers = [3, 1, 4, 1, 5]
print(sorted(numbers)) # [1, 1, 3, 4, 5]
```

You can also sort by a custom key:

```
words = ["banana", "apple", "cherry"]
print(sorted(words, key=len)) # ['apple', 'banana', 'cherry']
```

filter

The `filter` function constructs an iterator from elements of an iterable for which a function returns `True`:

```
numbers = [1, 2, 3, 4, 5]
even = filter(lambda x: x % 2 == 0, numbers)
print(list(even)) # [2, 4]
```

map

The `map` function applies a function to every item of an iterable and returns an iterator:

```
numbers = [1, 2, 3, 4]
squares = map(lambda x: x ** 2, numbers)
print(list(squares)) # [1, 4, 9, 16]
```

These functions are powerful tools for processing and transforming data in Python.

Modules & Packages in Python

Modules

A **module** is a single Python file (.py) containing definitions, functions, classes, or runnable code. Modules help organize code into reusable components.

Example:

```
# file: mymodule.py
def greet(name):
    print(f"Hello, {name}!")
```

Importing a module:

```
import mymodule
mymodule.greet("Alice")
```

Packages

A **package** is a directory containing multiple modules and a special `__init__.py` file (can be empty). Packages allow hierarchical structuring of modules.

Example structure:

```
mypackage/
  __init__.py
  module1.py
  module2.py
```

Importing from a package:

```
from mypackage import module1
module1.some_function()
```

Aliasing Modules & Imports

You can assign an alias to a module or object when importing, making it easier to reference in your code.

Aliasing a module:

```
import mymodule as mm  
mm.greet("Bob")
```

Aliasing an imported object:

```
from mymodule import greet as hello  
hello("Carol")
```

Aliasing is useful for shortening long module names or avoiding naming conflicts.

Exploring Modules with `help()` and `dir()`

Python provides built-in functions to explore modules and their contents:

- **`help()`**: Displays documentation for modules, functions, classes, or objects.
- **`dir()`**: Lists the names defined in a module or object (such as functions, classes, and variables).

Example usage:

```
import mymodule  
  
help(mymodule)           # Shows documentation for mymodule  
print(dir(mymodule))     # Lists all attributes and functions in  
                           mymodule
```

These tools are useful for discovering available functionality and understanding how to use modules.

Exploring Built-in Modules

Python comes with a rich set of built-in modules that provide standard functionality. You can explore available built-in modules using the `sys` and `pkgutil` modules.

Listing all built-in modules:

```
import sys

print(sys.builtin_module_names)
```

Finding all available modules (including installed ones):

```
import pkgutil

for module_info in pkgutil.iter_modules():
    print(module_info.name)
```

You can also use `help('modules')` in the Python interactive shell to see a list of all available modules.

These techniques help you discover modules that are ready to use without additional installation.

The `string` Built-in Module

Python's built-in `string` module provides useful constants and functions for string manipulation and formatting.

Common features:

- Predefined character sets like `ascii_letters`, `digits`, and `punctuation`.
- Utility functions for string formatting and templates.

Example usage:

```
import string

print(string.ascii_lowercase) # 'abcdefghijklmnopqrstuvwxyz'
print(string.digits)         # '0123456789'
```

The `string` module is helpful for tasks like validation, generating random strings, or custom formatting.

Exploring & Using the `math` Module

The built-in `math` module provides mathematical functions and constants, such as trigonometric functions, logarithms, and the value of `pi`.

Common features:

- Mathematical constants: `math.pi`, `math.e`
- Functions: `math.sqrt()`, `math.sin()`, `math.log()`, `math.factorial()`, etc.

Example usage:

```
import math

print(math.pi)           # 3.141592653589793
print(math.sqrt(16))     # 4.0
print(math.sin(math.pi)) # 0.0
print(math.log(100, 10)) # 2.0
```

Use `dir(math)` or `help(math)` to explore all available functions and constants in the module.

Error Handling & Debugging in Python

Error Handling

Python uses `try`, `except`, `else`, and `finally` blocks to handle exceptions (errors):

```
try:
    # Code that may raise an exception
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("No errors occurred.")
finally:
    print("This always runs.")
```

- **try:** Code that might throw an error.
- **except:** Handles specific exceptions.
- **else:** Runs if no exceptions occur.
- **finally:** Always runs, for cleanup.

Raising Exceptions

You can raise exceptions using `raise`:

```
if value < 0:
    raise ValueError("Value must be non-negative")
```

We can get the error information, error type & system message with `sys.exc_info()[0]` & `sys.exc_info()[1]`. We can have multiple `except` blocks for 1 `try` block.

Multiple Except Blocks

You can handle different exception types with multiple `except` blocks:

```
try:
    value = int(input("Enter a number: "))
    result = 10 / value
except ValueError:
    print("Invalid input! Please enter a number.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Each except block catches a specific error type.

Nested Try-Except Blocks

You can nest try-except blocks to handle errors at different levels:

```
try:
    value = int(input("Enter a number: "))
    try:
        result = 10 / value
        except ZeroDivisionError:
            print("Cannot divide by zero inside nested block!")
    except ValueError:
        print("Invalid input! Please enter a number.")
```

Nesting allows you to manage exceptions separately for different parts of your code.

Debugging

Using Print Statements

Insert `print()` statements to check variable values and program flow.

The pdb Debugger

Python includes a built-in debugger called `pdb`:

```
import pdb; pdb.set_trace()
```

This starts an interactive debugging session. (or)

```
python -m pdb <program_name.py>
```

Common pdb Commands

Command	Description
l (list)	Show source code around the current line
n (next)	Execute the next line (step over functions)
s (step)	Step into a function call
c (continue)	Continue execution until next breakpoint
b (break)	Set a breakpoint at a specified line
p (print)	Print the value of an expression
q (quit)	Quit the debugger and exit
h (help)	Show help for commands

Breakpoints

A breakpoint is a marker that tells the debugger to pause execution at a specific line, allowing you to inspect variables and program state.

Setting a Breakpoint in Code You can set a breakpoint in your code using `pdb.set_trace()`:

```
import pdb

def divide(a, b):
```

```
    pdb.set_trace()  # Execution will pause here
    return a / b

result = divide(10, 2)
print(result)
```

When the program reaches `pdb.set_trace()`, it enters interactive debugging mode.

Setting a Breakpoint in the Debugger Alternatively, you can set breakpoints by line number when running your script with `pdb`:

```
python -m pdb script.py
```

At the (Pdb) prompt, type:

```
b 5
```

This sets a breakpoint at line 5 of `script.py`. The program will pause at that line during execution.

Breakpoints help you stop execution at critical points to examine and debug your code effectively.

Managing Breakpoints in pdb

You can control breakpoints in the `pdb` debugger using several commands:

- **Listing Breakpoints**

To see all current breakpoints, use:

```
b
```

This will display a list of all breakpoints, their line numbers, and their status (enabled/disabled).

- **Enabling/Disabling Breakpoints**

If you have set breakpoints but want to temporarily disable or re-enable them:

```
disable <bpnumber>
```

```
enable <bpnumber>
```

Replace <bpnumber> with the breakpoint number shown in the list. Disabling a breakpoint means the debugger will not stop at that line until you enable it again.

- **Clearing Breakpoints**

To remove a specific breakpoint:

```
clear <bpnumber>
```

To remove all breakpoints at once:

```
clear
```

These commands help you manage where and when your program will pause during debugging, making it easier to focus on specific parts of your code.

Common Debugging Tools

- **IDEs:** Most IDEs (like VS Code, PyCharm) have integrated debuggers.
- **Logging:** Use the logging module for more advanced output than `print()`.

```
import logging
logging.basicConfig(level=logging.DEBUG)
logging.debug("This is a debug message")
```

File I/O in Python

File I/O (Input/Output) allows you to read from and write to files on your system.

Opening a File

Use the `open()` function:

```
file = open('example.txt', 'r') # 'r' for read mode
```

Common modes:

- 'r': Read (default)
- 'w': Write (creates/truncates file)
- 'a': Append
- 'b': Binary mode (e.g., 'rb')

Reading from a File

```
with open('example.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

Other methods:

- `readline()`: Reads one line
- `readlines()`: Reads all lines into a list

Writing to a File

```
with open('example.txt', 'w') as file:  
    file.write('Hello, world!\n')
```

Use 'a' mode to append instead of overwrite.

Closing a File

Files should be closed after use. Using `with` handles this automatically.

Example: Copying a File

```
with open('source.txt', 'r') as src, open('dest.txt', 'w') as dst:  
    for line in src:  
        dst.write(line)
```

Summary

- Use `open()` to access files.
- Always close files or use `with` for automatic handling.
- Choose the correct mode for your operation.
- Read and write using provided methods.

Object Oriented Programming (OOP) Concepts in Python

Object Oriented Programming (OOP) is a programming paradigm that organizes code using objects and classes. Python supports OOP, making it easier to structure complex programs.

Key Concepts

1. Class

A class is a blueprint for creating objects. It defines attributes and methods.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name} says woof!")
```

2. Object

An object is an instance of a class.

```
my_dog = Dog("Buddy")
my_dog.bark() # Output: Buddy says woof!
```

3. Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

```
class Animal:
    def eat(self):
        print("Eating...")
```

```
class Cat(Animal):  
    def meow(self):  
        print("Meow!")
```

4. Encapsulation

Encapsulation hides the internal state of an object and requires all interaction to be performed through methods.

```
class Counter:  
    def __init__(self):  
        self.__count = 0 # Private attribute  
  
    def increment(self):  
        self.__count += 1  
  
    def get_count(self):  
        return self.__count
```

5. Polymorphism

Polymorphism allows different classes to be treated as instances of the same class through a common interface.

```
class Bird:  
    def speak(self):  
        print("Chirp!")  
  
class Parrot(Bird):  
    def speak(self):  
        print("Squawk!")  
  
def animal_sound(animal):  
    animal.speak()
```

```
animal_sound(Bird())      # Output: Chirp!  
animal_sound(Parrot())    # Output: Squawk!
```

Summary

- **Class:** Blueprint for objects.
- **Object:** Instance of a class.
- **Inheritance:** Reuse code across classes.
- **Encapsulation:** Hide internal details.
- **Polymorphism:** Use a unified interface for different types.

OOP helps in organizing code, making it reusable and easier to maintain.

Advanced OOPs Concepts

Multiple Inheritance

Python supports multiple inheritance, where a class can inherit from multiple parent classes. The Method Resolution Order (MRO) determines the order in which methods are searched in the inheritance hierarchy.

```
class FlyingAnimal:
    def fly(self):
        print("I can fly!")

class SwimmingAnimal:
    def swim(self):
        print("I can swim!")

class Duck(FlyingAnimal, SwimmingAnimal):
    def quack(self):
        print("Quack!")

duck = Duck()
duck.fly()
duck.swim()
duck.quack()

# Understanding MRO
print(Duck.__mro__)
# Output: (<class '__main__.Duck'>, <class
#         '__main__.FlyingAnimal'>, <class '__main__.SwimmingAnimal'>,
#         <class 'object'>)
```

Polymorphism with Abstract Base Classes (ABCs)

Polymorphism allows objects of different classes to be treated as objects of a common type. While regular inheritance provides this, Abstract Base Classes (ABCs) from the abc module enforce that certain methods must be implemented by subclasses, acting as blueprints or interfaces.

```
from abc import ABC, abstractmethod

class Shape(ABC): # Inherit from ABC to make it an abstract base
    class
    @abstractmethod
    def area(self):
        """Calculates the area of the shape."""
        pass

    @abstractmethod
    def perimeter(self):
        """Calculates the perimeter of the shape."""
        pass

    def description(self):
        """A concrete method that can be inherited as is."""
        return "This is a geometric shape."

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14159 * self.radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# This would raise a TypeError because Shape is abstract
# s = Shape()
```

```
circle = Circle(5)
rectangle = Rectangle(4, 6)

shapes = [circle, rectangle]

for shape in shapes:
    print(f"Shape: {shape.__class__.__name__}")
    print(f"Area: {shape.area():.2f}")
    print(f"Perimeter: {shape.perimeter():.2f}")
    print(f>Description: {shape.description()}\n")
```

Why use ABCs?

- Enforce Contracts: Ensure that subclasses provide specific implementations for abstract methods.
- Prevent Instantiation: You cannot create instances of an abstract class directly.
- Clearer Design: Communicate the intended interface for a family of classes.

Dunder Methods

Special methods (also known as "dunder" methods because of their double underscores, e.g., **init**, **str**) allow you to define how objects of your class behave in response to built-in operations, functions, and language constructs.

Advanced Python Concepts

Generators

Generators are special functions that yield values one at a time using the `yield` keyword. They allow you to iterate over large data sets without loading everything into memory.

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
for num in countdown(5):  
    print(num)
```

Advantages:

- Memory efficient
 - Lazy evaluation (values produced on demand)
-

Iterators

An iterator is an object that implements the `__iter__()` and `__next__()` methods. Iterators allow you to traverse through all the elements of a collection.

```
class Counter:  
    def __init__(self, low, high):  
        self.current = low  
        self.high = high  
  
    def __iter__(self):  
        return self
```

```
def __next__(self):
    if self.current > self.high:
        raise StopIteration
    else:
        self.current += 1
        return self.current - 1

for num in Counter(1, 3):
    print(num)
```

Decorators

Python's decorators are a powerful and elegant way to modify or enhance the behavior of functions and methods (which are functions within a class). They essentially wrap another function or method, providing additional functionality before or after the original code executes.

1. Function Decorators

Function decorators are used to modify the behavior of individual functions.

Consider a simple function

```
def greet(name):
    return f"Hello, {name}!"
```

let's say we want to add a logging feature to this function, printing when it's called and with what arguments.

```
def my_logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function '{func.__name__}' with args:
              {args}, kwargs: {kwargs}")
        result = func(*args, **kwargs)
        print(f"Function '{func.__name__}' finished. Result:
              {result}")
    return wrapper
```

```
        return result
    return wrapper

# Manually decorating the greet function
greet = my_logger(greet)

print(greet("Alice"))
```

Using the @ Syntax (Syntactic Sugar)

Common Use Cases for Function Decorators

1. **Logging:** Recording function calls, arguments, and return values.

```
def my_logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function '{func.__name__}' with args:
              ↳ {args}, kwargs: {kwargs}")
        result = func(*args, **kwargs)
        print(f"Function '{func.__name__}' finished. Result:
              ↳ {result}")
        return result
    return wrapper

@my_logger
def greet(name):
    return f"Hello, {name}!"

@my_logger
def add(a, b):
    return a + b

print(greet("Bob"))
print("-" * 20)
print(add(10, 20))
```

2. **Timing:** Measuring the execution time of a function.

```
import time

def timing_decorator(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"Function '{func.__name__}' took {end_time -
            start_time:.4f} seconds to execute.")
        return result
    return wrapper

@timing_decorator
def long_running_task():
    time.sleep(2)
    return "Task completed!"

print(long_running_task())
```

3. Authentication/Authorization: Checking if a user has the necessary permissions before executing a function.

```
def requires_admin(func):
    def wrapper(user, *args, **kwargs):
        if user != "admin":
            raise PermissionError("Access denied: Admin privileges
                required.")
        return func(user, *args, **kwargs)
    return wrapper

@requires_admin
def delete_sensitive_data(user, data_id):
    return f"User '{user}' deleted data {data_id}."

try:
    print(delete_sensitive_data("admin", 123))
    print(delete_sensitive_data("guest", 456))
except PermissionError as e:
    print(e)
```

4. Caching: Storing the results of expensive function calls and returning the cached result if the same arguments are provided again.

```
from functools import lru_cache

@lru_cache(maxsize=None) # A built-in Python decorator for caching
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10)) # This will compute and cache
print(fibonacci(10)) # This will return from cache immediately
```

5. Input Validation: Checking the types or values of arguments
6. Retries: Automatically retrying a function if it fails.

Decorators with Arguments

Sometimes, you need to pass arguments to your decorator. This requires an extra layer of nesting.

```
def repeat(num_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(num_times=3)
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Charlie")
```

2. Class Decorators Class decorators are used to modify or replace an entire class. They take a class object as an argument and return either a new class or the modified original class.

Simple Class Decorator

```
def add_attribute(cls):
    cls.MY_ATTRIBUTE = 5
    return cls

@add_attribute
class MyClass:
    def __init__(self, value):
        self.value = value

print(MyClass.MY_ATTRIBUTE)

obj = MyClass(10)
print(obj.value)
```

Class Decorator that Replaces the Class

A class decorator can also return an entirely new class, effectively replacing the original.

```
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance # Return a function that acts like the
    ↪ class constructor

@singleton
class DatabaseConnection:
    def __init__(self):
        print("Creating a new database connection...")
        self.connection_id = id(self) # Unique ID for this
        ↪ connection

    def connect(self):
        return f"Connected with ID: {self.connection_id}"
```

```
# Both obj1 and obj2 will be the same instance
obj1 = DatabaseConnection()
obj2 = DatabaseConnection()

print(obj1.connect())
print(obj2.connect())
print(obj1 is obj2)
```

Class Decorator with Arguments

Similar to function decorators, if your class decorator needs arguments, you add an extra layer.

```
def register_class(registry_name):
    def decorator(cls):
        print(f"Registering class '{cls.__name__}' in registry  

↳ '{registry_name}'")
        # In a real scenario, you'd store cls in a global registry  

↳ based on registry_name
        return cls
    return decorator

@register_class(registry_name="api_models")
class UserModel:
    def __init__(self, name):
        self.name = name

@register_class(registry_name="data_processors")
class DataProcessor:
    def process(self, data):
        return f"Processing {data}"
```

Common Use Cases for Class Decorators:

1. **Singleton Pattern:** Ensuring that only one instance of a class exists (as shown above).
2. **Adding Mixins:** Programmatically injecting methods or attributes into a class.

3. **Registering Classes:** Automatically registering classes in a central registry for plugins, factories, or configuration.
4. **Interface Enforcement:** Checking if a class implements certain methods or attributes.
5. **Meta-programming:** Dynamically modifying class behavior, adding methods, or changing method implementations.
6. **ORMs (Object-Relational Mappers):** Often use class decorators to map Python classes to database tables.

Key Considerations and Best Practices

- `functools.wraps`: When creating function decorators, it's crucial to use `functools.wraps` from the standard library. This helper decorator preserves the original function's metadata (like `__name__`, `__doc__`, `__module__`, etc.) of the decorated function. Without it, debugging and introspection can become difficult.

```
from functools import wraps
import time

def timing_decorator(func):
    @wraps(func) # Use this!
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"Function '{func.__name__}' took {end_time -
            start_time:.4f} seconds to execute.")
        return result
    return wrapper

@timing_decorator
def my_function():
    """This is a docstring for my_function."""
    time.sleep(1)
    return "Done"

print(my_function.__name__) # Without @wraps, this would be
    'wrapper'
```

```
print(my_function.__doc__) # Without @wraps, this would be None
my_function()
```

- **Readability:** While powerful, excessive use of decorators or overly complex decorators can make code harder to read and understand. Use them judiciously.
- **Order of Decorators:** When multiple decorators are applied to a single function or class, they are applied from bottom to top.
- **Debugging:** Decorators add layers to your code execution. Tools like `pdb` might require a bit more navigation when stepping through decorated functions.

Dataclasses

Dataclasses, introduced in Python 3.7, provide a decorator and functions for automatically adding special methods such as `__init__()` and `__repr__()` to user-defined classes.

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p = Point(1, 2)
print(p) # Output: Point(x=1, y=2)
```

Benefits:

- Reduces boilerplate code
- Provides built-in methods for comparison and representation

List Comprehension

List comprehensions offer a concise way to create lists using a single line of code.

```
squares = [x**2 for x in range(5)]  
print(squares) # Output: [0, 1, 4, 9, 16]
```

Advantages:

- More readable and compact than traditional loops
 - Can include conditions: `[x for x in range(10) if x % 2 == 0]`
-

Dictionary Comprehension

Dictionary comprehensions allow you to construct dictionaries in a clear and concise way.

```
squares = {x: x**2 for x in range(5)}  
print(squares) # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Use cases:

- Transforming or filtering dictionary data
- Creating mappings from sequences

`__slots__`

The `__slots__` declaration in a Python class is used to explicitly declare data members and prevent the creation of a dynamic `__dict__` for each instance. This can save memory and improve attribute access speed, especially when creating many instances.

```
class Person:
    __slots__ = ['name', 'age']

    def __init__(self, name, age):
        self.name = name
        self.age = age

p = Person("Alice", 30)
print(p.name)
```

Benefits:

- Reduces memory usage by avoiding per-instance dictionaries
- Prevents accidental creation of new attributes

Limitations:

- Cannot add attributes not listed in `__slots__`
- May not work well with multiple inheritance

Context Managers

Context managers in Python provide a convenient and robust way to manage resources, ensuring that setup actions (like opening a file or acquiring a lock) are properly handled and corresponding teardown actions (like closing the file or releasing the lock) are always executed, even if errors occur during the resource's usage.

The core of context managers is the `with` statement.

Consider a common scenario: file handling. If you open a file, you must close it, regardless of whether reading/writing to it succeeds or fails.

Without a context manager (prone to errors)

```
file = open("my_file.txt", "w")
try:
    file.write("Hello, World!")
    # What if an error occurs here? The file might not be closed.
```

```
# For example, if we try to divide by zero:  
# result = 1 / 0  
except Exception as e:  
    print(f"An error occurred: {e}")  
finally:  
    file.close() # This ensures it's closed, but it's verbose.
```

The with statement simplifies this by abstracting the setup and teardown logic.

```
with open("my_file.txt", "w") as file:  
    file.write("Hello, World!")  
    # Even if an error occurs here, the file will be automatically  
    closed.  
    # result = 1 / 0 # Try uncommenting this to see the  
    auto-closing in action  
print("File operations complete.")
```

Explanation of the with statement:

1. When the with statement is encountered, Python calls a special method on the context manager object: **enter()**.
2. The return value of **enter()** (if any) is assigned to the variable after as (e.g., file in the example).
3. The code inside the with block is then executed.
4. Once the with block is exited (either normally or due to an exception), another special method is called: **exit()**. This method handles the cleanup.

How to Create Your Own Context Managers There are two primary ways to create context managers:

1. Using Classes: By implementing the **enter** and **exit** methods.
2. Using the contextlib module: A simpler way for functions using yield.

Method 1: Using Classes (Implementing **enter** and **exit**)

To make an object a context manager, its class must define both **enter** and **exit** methods.

Context managers in Python provide a convenient and robust way to manage resources, ensuring that setup actions (like opening a file or acquiring a lock) are properly handled and corresponding teardown actions (like closing the file or releasing the lock) are always executed, even if errors occur during the resource's usage.

The core of context managers is the with statement.

The Problem Context Managers Solve

Consider a common scenario: file handling. If you open a file, you must close it, regardless of whether reading/writing to it succeeds or fails.

Without a context manager (prone to errors):

```
file = open("my_file.txt", "w")
try:
    file.write("Hello, World!")
    # What if an error occurs here? The file might not be closed.
    # For example, if we try to divide by zero:
    # result = 1 / 0
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    file.close() # This ensures it's closed, but it's verbose.
```

This try...finally block is a common pattern for resource management, but it can be verbose and repetitive, especially when dealing with multiple resources.

Introducing the with Statement and Context Managers

The with statement simplifies this by abstracting the setup and teardown logic.

With a context manager (elegant and safe):

```
with open("my_file.txt", "w") as file:
    file.write("Hello, World!")
    # Even if an error occurs here, the file will be automatically
    # closed.
```

```
# result = 1 / 0 # Try uncommenting this to see the
    auto-closing in action
print("File operations complete.")
```

Explanation of the with statement:

- When the with statement is encountered, Python calls a special method on the context manager object: `__enter__()`.
- The return value of `__enter__()` (if any) is assigned to the variable after as (e.g., `file` in the example).
- The code inside the with block is then executed.
- Once the with block is exited (either normally or due to an exception), another special method is called: `__exit__()`. This method handles the cleanup.

How to Create Your Own Context Managers

There are two primary ways to create context managers:

- Using Classes: By implementing the `__enter__` and `__exit__` methods.
- Using the `contextlib` module: A simpler way for functions using `yield`.

Method 1: Using Classes (Implementing `__enter__` and `__exit__`)

To make an object a context manager, its class must define both **enter** and **exit** methods.

Example: A custom file handler

```
class MyFileHandler:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None # Initialize to None

    def __enter__(self):
        print(f"Entering context: Opening '{self.filename}' in mode
        '{self.mode}'")
        self.file = open(self.filename, self.mode)
        return self.file
```

```
def __exit__(self, exc_type, exc_val, exc_tb):
    # exc_type, exc_val, exc_tb contain information about an
    # exception, if one occurred
    if self.file:
        self.file.close()
        print(f"Exiting context: Closed '{self.filename}'")
    if exc_type:
        print(f"An exception occurred:
        ↪ Type={exc_type.__name__}, Value={exc_val}")
        # Returning True from __exit__ suppresses the
        ↪ exception.
        # Returning False (or nothing) re-raises the exception.
        return False # Let the exception propagate

print("--- Using MyFileHandler with no error ---")
with MyFileHandler("example.txt", "w") as f:
    f.write("This is a test line.\n")
    f.write("Another line.\n")

print("\n--- Using MyFileHandler with an error ---")
try:
    with MyFileHandler("error_test.txt", "w") as f:
        f.write("This line will be written.\n")
        raise ValueError("Something went wrong inside the block!")
        f.write("This line will NOT be written.\n") # Unreachable
except ValueError as e:
    print(f"Caught the error: {e}")

# Verify file content
with open("example.txt", "r") as f:
    print("\nContent of example.txt:")
    print(f.read())
```

Method 2: Using the contextlib Module (Decorator @contextmanager)

The contextlib module provides the @contextmanager decorator, which allows you to create context managers using a simple generator function. This is often more concise and readable for simpler cases.

Example: A context manager for database connections

```
from contextlib import contextmanager
import sqlite3

@contextmanager
def db_connection(db_name):
    print(f"Entering context: Connecting to database  
↳ '{db_name}'...")
    conn = None
    try:
        conn = sqlite3.connect(db_name)
        yield conn # This is where the 'with' block's code is  
↳ executed
    except sqlite3.Error as e:
        print(f"Database error: {e}")
        raise # Re-raise the exception after logging
    finally:
        if conn:
            conn.close()
            print(f"Exiting context: Closed database connection to  
↳ '{db_name}'")

print("--- Using db_connection with no error ---")
with db_connection("my_database.db") as conn:
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER  
↳ PRIMARY KEY, name TEXT)")
    cursor.execute("INSERT INTO users (name) VALUES (?)",  
↳ ("Alice",))
    cursor.execute("INSERT INTO users (name) VALUES (?)", ("Bob",))
    conn.commit()
    print("Users added.")

print("\n--- Using db_connection with a simulated error ---")
try:
    with db_connection("my_database.db") as conn:
        cursor = conn.cursor()
        cursor.execute("INSERT INTO users (name) VALUES (?)",  
↳ ("Charlie",))
        print("This line will be printed.")
        raise ValueError("Simulated error during DB operation!")
    conn.commit() # This won't be reached
```

```
except ValueError as e:
    print(f"Caught the simulated error: {e}")

# Verify data (should only have Alice, Bob, and Charlie if error
# was caught)
with db_connection("my_database.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users")
    print("\nUsers in database:")
    for row in cursor.fetchall():
        print(row)
```

Common Use Cases for Context Managers

1. File I/O: (Most common example) Automatically closing files.

```
with open('data.txt', 'r') as f:
    content = f.read()
```

2. Locking and Thread Synchronization: Acquiring and releasing locks.

```
import threading

lock = threading.Lock()
with lock: # Acquires the lock on __enter__, releases on __exit__
    # Critical section of code
    print("Accessing shared resource safely.")
```

3. Database Connections: Opening and closing database connections.
4. Networking Sockets: Opening and closing network connections.
5. Temporary Resources: Creating a temporary directory or file that is automatically cleaned up

```
import tempfile
import os

with tempfile.TemporaryDirectory() as tmpdir:
    print(f"Created temporary directory: {tmpdir}")
```

```
file_path = os.path.join(tmpdir, "temp_data.txt")
with open(file_path, "w") as f:
    f.write("Temporary data.")
# tmpdir and file_path will be removed when exiting the with
    block
print("Temporary directory removed.")
```